TỜ HƯỚNG DẪN SỬ DỤNG

Grid Search trọng số cho Bloxorz Solver

Quy trình tái lập cho UCS • A • Greedy Best-First Search (GBFS)*

Mục tiêu của tài liệu

Hướng dẫn nhóm tự chạy thí nghiệm trên bộ 18 level, lưu kết quả CSV/JSON, lựa chọn tham số theo tiêu chí công bố trước và viết phần phương pháp/kết quả có thể tái lập. Tài liệu này không chứa kết quả Grid Search đã chạy trên máy của nhóm.

Bản chất của thí nghiệm

Đây là tuning theo một rubric kỹ thuật được nhóm định nghĩa trước, không phải học trọng số từ nhãn sở thích người chơi. Dataset hiện có các quỹ đạo sinh từ solver và điểm đánh giá cũ; các nhãn score/rating cũ không được xem là ground truth độc lập.

Không gọi kết quả là "bộ trọng số tối ưu phổ quát". Cách diễn đạt đúng là: "cấu hình tốt nhất trong lưới đã khảo sát, theo tiêu chí đã định trước, trên tập level và giao thức thực nghiệm này".

Bộ kit có gì?

Thành phần	Vai trò
experiments/tune_weights.py	Script Grid Search, audit quỹ đạo, kiểm tra heuristic, benchmark lặp, xuất CSV/JSON/PNG.
benchmark_levels/levels/	18 level JSON dùng làm tập test cố định.
benchmark_levels/trajectories/	180 bản ghi quỹ đạo được audit; không dùng score/rating cũ làm nhãn học.
Bloxorz_Solver-.../src/core/	Bản board.py và state.py dùng để gọi logic Board/State/transition thật.
templates/	Mẫu header CSV và bảng báo cáo.

Lưu ý về tích hợp

Script chạy một search harness riêng để nghiên cứu trọng số; nó không tự sửa các lớp solver trong GUI. Sau khi chọn tham số, nhóm phải tích hợp và kiểm thử riêng nếu muốn GUI sử dụng đúng cấu hình mới.

1. Kiểm tra dataset trước khi chạy

Bộ dữ liệu gồm 18 level và 180 bản ghi quỹ đạo. Các chuỗi action trùng nhau trong cùng level được ghi nhận trong file audit; chúng không nên được xem là các ví dụ độc lập. Đặc biệt, trong Level 16 có năm bản ghi không tới đích nhưng score/rating lưu sẵn vẫn cao. Script sẽ audit replay và không dùng những score/rating đó để lựa chọn trọng số.

Bước audit không đồng nghĩa phải xóa dữ liệu gốc. Hãy giữ file gốc để truy vết, dùng trajectory_audit.csv để xác định bản ghi thành công, thất bại, không hợp lệ và trùng action sequence.

2. Mô hình chi phí UCS

Cố định cost cơ bản ở 1.0 cho mỗi lần lăn U/D/L/R. SPACE/SWITCH chỉ đổi cube đang điều khiển, không tính là nước đi theo FAQ giảng viên và có cost cơ bản 0. Hệ số cho mỗi lần lăn được cố định để loại bỏ nhập nhằng do nhân mọi cost với cùng hằng số dương.

Công thức transition cost trong harness:

$$c(s, a, s') = I[a \in \{U,D,L,R\}] + w_b B + w_v V + w_f F + w_m M + w_c C$$

Tổng cost của một quỹ đạo là tổng cost của từng transition. Action lăn U/D/L/R có cost cơ bản 1; SPACE/SWITCH đổi cube điều khiển có cost cơ bản 0 và không phát sinh penalty di chuyển. Các cost đều không âm.

Feature	Định nghĩa vận hành của script	Giới hạn diễn giải
S — steps	Chỉ action lăn U/D/L/R được tính là một nước đi và có cost cơ bản 1. SPACE/SWITCH đổi cube điều khiển, cost cơ bản 0.	Chỉ đếm lần lăn U/D/L/R, không đếm trạng thái; SPACE/SWITCH lưu riêng.
B — backtrack	Một lần lăn đảo ngược ngay lần lăn trước đó trong chuỗi chuyển động.	Khóa tìm kiếm lưu (state, last_action); SPACE/SWITCH reset ngữ cảnh vì đổi cube điều khiển.
V — void exposure	Một điểm cho transition nếu có ô đang bị chiếm giữ với ≥2 hướng lân cận bị chặn/ngoài biên/không hợp lệ.	Proxy địa hình, không phải xác suất rơi được đo.
F — fragile entry	Số ô fragile mới đi vào trong transition.	Tiếp xúc/đi vào, không mặc nhiên là lỗi; khối nằm ngang có thể hợp lệ.
M — switch-change proxy	Số cờ bridge đổi trạng thái trong transition.	Proxy quan sát cấu hình cầu; không nhận diện hoàn hảo mọi switch dư thừa.
C — bridge entry	Số ô bridge mới mà khối đi vào.	Không đồng nghĩa số lần kích hoạt switch.

Baseline ban đầu của nhóm: w_backtrack=3, w_void=2, w_fragile=4, w_switch_change=2.5, w_bridge_entry=3.

Lưới UCS đã định trước (3^5 = 243 cấu hình, bao gồm baseline)

Tham số	Các giá trị khảo sát
w_backtrack	{0, 3, 5}
w_void	{0, 2, 4}
w_fragile	{0, 4, 8}
w_switch_change	{0, 2.5, 5}
w_bridge_entry	{0, 3, 5}

3. Tiêu chí chọn trọng số UCS

Không chọn cấu hình chỉ vì nó đạt tổng cost thấp nhất theo chính hàm cost của nó; UCS được thiết kế để tối thiểu hóa đại lượng đó. Harness dùng rubric kỹ thuật nhiều tiêu chí được công bố trước và không sử dụng các trường score/rating cũ của JSON làm nhãn.

Với từng level, feature của đường đi do các cấu hình khác nhau tạo ra được đổi thành percentile rank trong nhóm ứng viên của chính level đó. Rank thấp hơn nghĩa là feature có giá trị thấp hơn tương đối trong nhóm ứng viên. Loss gộp:

L = (r_S + 3 r_B + 2 r_V + 4 r_F + 2.5 r_M + 3 r_C) / 15.5

Các hệ số là mức ưu tiên theo rubric kỹ thuật của nhóm, lấy tỷ lệ từ các hệ số baseline. Đây là một lựa chọn thiết kế định trước, không phải trọng số được suy ra từ ground truth con người.

Nếu đường đi thất bại hoặc replay không hợp lệ, script gán loss = 2.0 để cấu hình đó không được ưu tiên hơn một nghiệm hợp lệ. Với các nghiệm hợp lệ, mỗi feature được xếp hạng riêng trong từng level trước khi tính loss.

Quy tắc quyết định UCS

Trong mỗi fold, chọn cấu hình có loss trung bình thấp nhất trên các level train. So sánh trên các level held-out với baseline gốc. Chỉ thay baseline nếu loss held-out trung bình cải thiện; nếu không cải thiện, giữ bộ trọng số ban đầu.

4. Chia dữ liệu: Grouped 6-fold theo level_id

Không chia ngẫu nhiên các quỹ đạo cùng level vào cả train và test, vì chúng không độc lập. Script dùng sáu fold cố định, mỗi fold giữ lại ba level. Cấu hình được chọn bằng các level còn lại; các level held-out chỉ được dùng để đánh giá độ ổn định.

Fold	Level held-out
1	level_01, level_09, level_15
2	level_02, level_10, level_16
3	level_03, level_05, level_11
4	level_04, level_06, level_12
5	level_07, level_13, level_17
6	level_08, level_14, level_18

Đây là đánh giá ổn định theo nhóm level, chưa phải kiểm thử độc lập trên những level hoàn toàn mới được thu thập từ bên ngoài. Hãy nêu giới hạn này trong báo cáo.

5. Tuning A*: giữ cost model, chỉnh heuristic

A* dùng f(s) = g(s) + h(s). Để so sánh công bằng với UCS, g(s) phải dùng đúng cost model đã được chọn ở giai đoạn UCS; phần Grid Search của A* chỉ thay đổi hệ số scale của heuristic.

$h_\alpha(s) = \alpha\, h_0(s)$, với $\alpha \in \{0, 0.25, 0.5, 0.75, 1.0\}$

Trên board không có split switch, h_0 được định nghĩa là khoảng cách Manhattan nhỏ nhất từ các ô khối đang chiếm giữ tới goal, chia 2, cộng 0.5 nếu khối không standing. Trên board có split switch hoặc trạng thái split, $h_0 = 0$ để tránh sử dụng chặn hình học không an toàn trước cơ chế dịch chuyển.

- α = 0 là mốc không định hướng, có thứ tự ưu tiên theo g; tương đương UCS theo cost trên cùng state graph.
- Chỉ chấp nhận cấu hình A* khi đường đi replay hợp lệ và tổng cost bằng chi phí tối ưu UCS trên cùng level và cùng cost model.
- Trong các α hợp lệ, script ưu tiên số node mở rộng thấp; thời gian là tiêu chí phụ vì dễ nhiễu.
- Kiểm tra admissibility/consistency trên 18 graph chỉ là bằng chứng kiểm thử trên dataset này, không phải chứng minh cho mọi board có thể tạo.

6. Tuning GBFS: heuristic riêng

GBFS ưu tiên node chỉ theo h(s), không cộng g(s), vì vậy không đảm bảo đường đi tối ưu theo tổng cost. Heuristic của harness là:

$h(s) = d(s) + w_o\, O(s) + w_p\, P(s) + w_e\, E(s) + w_f\, F(s)$

d(s) là khoảng cách thư giãn tới đích (hệ số cố định bằng 1); O là chỉ báo khối chưa standing; P là chỉ báo trạng thái split; E là số ô đang chiếm giữ bị tính là sát vực; F là số ô fragile đang bị chiếm giữ.

Tham số GBFS	Các giá trị khảo sát
w_orientation	{0, 0.5, 1, 1.5}
w_split	{0, 1, 2, 3}
w_edge	{0, 0.5, 1, 2}
w_fragile	{0, 1, 2, 4}

Số cấu hình GBFS

4 × 4 × 4 × 4 = 256 cấu hình. Baseline (1, 1, 0, 0) phản ánh các khoản phạt tư thế/split trong heuristic hiện tại của GreedySolver. Hệ số toàn cục nhân chung toàn bộ heuristic không được tìm vì thường không làm thay đổi thứ tự ưu tiên.

GBFS chọn cấu hình theo rubric chất lượng và kết quả trên các fold held-out. Nếu không cải thiện baseline trên held-out, script giữ heuristic baseline. Không diễn giải GBFS như thuật toán tối ưu cost.

7. Hướng dẫn chạy trên Windows — làm theo đúng thứ tự

> ***Nếu bạn đã chạy bản starter kit cũ, hãy sao lưu rồi xóa các file trong experiments/results/ trước lần chạy V2 đầu tiên. Checkpoint cũ được tạo khi SPACE còn bị tính như một move nên không được trộn với kết quả mới.***

Bước 0 — Tải và giải nén Bloxorz_Weight_Tuning_Starter_Kit.zip vào đường dẫn đơn giản, không dùng thư mục có dấu, ví dụ:

```
D:\AI_Project\Bloxorz_Weight_Tuning_Guide
```

Giữ nguyên cấu trúc thư mục: benchmark_levels, experiments, templates, và Bloxorz_Solver-feat-team-code-phase2-game-gui.

Bước 1 — Mở PowerShell tại thư mục gốc

Trong File Explorer, mở thư mục vừa giải nén, nhấp thanh địa chỉ, gõ powershell rồi nhấn Enter. Kiểm tra vị trí hiện tại bằng pwd.

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python --version
```

Yêu cầu Python 3.10 trở lên và thư viện matplotlib. Nếu PowerShell chặn kích hoạt môi trường, dùng Command Prompt và lệnh .venv\Scripts\activate.bat.

Bước 2 — Kiểm tra script có nhận đúng tùy chọn

```
python experiments\tune_weights.py --help
```

Script tự tìm 18 JSON trong benchmark_levels/levels, quỹ đạo trong benchmark_levels/trajectories và ghi đầu ra vào experiments/results/.

Bước 3 — Chạy toàn bộ quy trình (khuyên dùng)

```
python experiments\tune_weights.py --phase all --repeats 5
```

Lệnh này audit quỹ đạo, xây graph từ Board.transition, quét UCS, tuning alpha A*, tuning GBFS và benchmark lặp cấu hình được chọn 5 lần trên mỗi level. Không đóng cửa sổ terminal khi script còn chạy.

Chạy từng giai đoạn (chỉ khi cần)

```
python experiments\tune_weights.py --phase ucs
python experiments\tune_weights.py --phase astar
python experiments\tune_weights.py --phase gbfs
python experiments\tune_weights.py --phase final --repeats 5
```

Chạy theo đúng thứ tự trên, vì A* cần kết quả UCS; final cần kết quả UCS, A* và GBFS. Dễ nhất vẫn là chạy một lần --phase all --repeats 5.

Nếu bị gián đoạn

UCS có checkpoint ucs_grid_raw_checkpoint.csv; chạy lại cùng lệnh có thể tiếp tục những config/level chưa hoàn tất. Nếu muốn thí nghiệm mới hoàn toàn, hãy sao lưu kết quả cũ rồi xóa các file trong experiments/results/ trước khi chạy. Không xóa checkpoint nếu muốn resume phiên chạy cũ.

8. File đầu ra và cách sử dụng

File đầu ra	Ý nghĩa / dùng trong báo cáo
trajectory_audit.csv	Kiểm tra quỹ đạo cũ: replay thành công/thất bại, chuỗi action trùng, score/rating đã lưu. Dùng để mô tả làm sạch dữ liệu.
graph_stats.json	Số state/cạnh đạt tới theo level và thời gian xây graph.
ucs_grid_results.csv	Kết quả chi tiết từng cấu hình UCS × level, feature, cost, số node.
ucs_grouped_cv.csv	Cấu hình chọn trong từng fold, kết quả train/held-out so với baseline.
ucs_top_configs.csv	Các cấu hình UCS tốt theo loss rubric trên lưới đã khảo sát.
ucs_tuning_summary.json	Trọng số UCS được chọn và quyết định giữ/thay baseline.
astar_alpha_cv_grid.csv, astar_heuristic_checks.csv	Kết quả tuning alpha A* và kiểm tra heuristic trên state graph.
astar_alpha_summary.csv, astar_tuning_summary.json	So sánh alpha và cấu hình A* được chọn.
gbfs_grid_results.csv, gbfs_grouped_cv.csv	Kết quả chi tiết lưới GBFS và so sánh held-out.
gbfs_top_configs.csv, gbfs_tuning_summary.json	Các cấu hình GBFS ứng viên và tham số được chọn.
final_selected_weights_repeated_benchmark.csv	Dữ liệu gốc từng lần benchmark lặp.
final_selected_weights_summary.csv	Thống kê tổng hợp thời gian, bộ nhớ, số node, số action và tổng cost.
experiment_summary.json, METHODS_AND_LIMITATIONS.md	Cấu hình được chọn, định nghĩa feature, môi trường và giới hạn phương pháp.
Các file .png	Biểu đồ so sánh UCS, A* và GBFS được tạo từ kết quả CSV.

9. Cách chọn và diễn giải cấu hình

UCS

- Nếu mean held-out quality loss cải thiện so với baseline, báo cáo mức cải thiện và kết quả từng fold.
- Nếu không cải thiện, giữ baseline (3, 2, 4, 2.5, 3). Kết quả không cải thiện vẫn hợp lệ; không chọn cấu hình chỉ vì có total cost thấp nhất.

A*

- Giữ cost model giống UCS, chỉ thay alpha. Kiểm tra route replay hợp lệ và total cost khớp UCS.
- Báo cáo số node mở rộng cùng median thời gian; nêu h = 0 trên các board có split-switch.

GBFS

- Chọn theo rubric và kết quả held-out, không chỉ dựa trên thời gian hoặc node.
- GBFS không bảo đảm tối ưu; luôn báo cáo success, solution_valid, solution_length và feature đường đi.

10. Số liệu cần lấy để ghi vào báo cáo

Tổng hợp từ CSV/JSON, không dùng ảnh terminal làm nguồn chính. Với mỗi cấu hình cuối, báo cáo các chỉ số sau:

Chỉ số	Cách báo cáo
Success rate	Số level tìm được nghiệm hợp lệ / tổng số level.
Search time (ms)	Median và IQR qua các lần chạy; ghi rõ đo trong search harness, không gồm GUI/animation.
Peak memory (KiB)	Median và IQR của bộ nhớ Python được tracemalloc theo dõi; không gọi là tổng RAM tiến trình.
Expanded nodes	Median theo level và quy tắc đếm trong script.

Chỉ số	Cách báo cáo
Solution length	Số lần lăn U/D/L/R; SPACE/SWITCH không tính là nước đi. Báo cáo action_count và control_switches riêng; thất bại là N/A.
Total path cost	Bắt buộc với UCS/A* cùng cost model; không diễn giải GBFS như tối ưu cost.
Held-out quality loss	So sánh tuned với baseline theo cùng rubric và từng fold.
Validity	Replay toàn bộ action; chỉ tính nghiệm hợp lệ nếu kết thúc đúng goal.

11. Đoạn phương pháp có thể đưa vào báo cáo (điền kết quả thật)

Nhóm dùng Grid Search trên lưới tham số được xác định trước. Chi phí cơ bản của mỗi lần lăn U/D/L/R được cố định ở 1.0; SPACE/SWITCH chỉ đổi cube điều khiển, không tính là nước đi và có chi phí cơ bản 0. Năm trọng số phạt được khảo sát trên 243 cấu hình, gồm baseline. Việc chọn cấu hình dựa trên rubric kỹ thuật đa tiêu chí với percentile rank của feature theo từng level; score/rating cũ không được dùng làm ground truth. Nhóm áp dụng sáu fold theo level_id để các quỹ đạo cùng level không nằm đồng thời trong train và held-out. Chỉ thay baseline nếu loss held-out cải thiện theo quy tắc định trước. A* giữ cost model UCS và tuning alpha; GBFS tuning heuristic riêng, không tuyên bố tối ưu đường đi. Vì chưa có nhãn người dùng độc lập, đây là tuning kỹ thuật theo rubric, không phải học có giám sát từ ground truth.

Điền bộ trọng số được chọn, kết quả held-out và mức cải thiện so với baseline. Báo cáo success rate, median/IQR thời gian và bộ nhớ, expanded nodes, số lần lăn U/D/L/R, total cost, biểu đồ và hạn chế. Chỉ dùng số lấy từ output.

12. Những câu không nên tuyên bố

- Không tuyên bố tối ưu tuyệt đối; chỉ kết luận trong lưới/rubric đã khảo sát. Không dùng score/rating cũ làm ground truth độc lập.
- GBFS không bảo đảm tối ưu cost; không phát hiện vi phạm heuristic trên 18 level không chứng minh đúng cho mọi board.
- Với cost phi đồng nhất, không chỉ so sánh số lần lăn; báo cáo total path cost và tính hợp lệ.

13. Checklist trước khi chốt số liệu

☐ Giải nén kit; ghi phiên bản Python.	☐ Giữ bản gốc dataset/source; cố định level.
☐ Chạy `--phase all --repeats 5`.	☐ Rà `trajectory_audit.csv` (thất bại/trùng).
☐ Lưu CSV/JSON/PNG và log gốc.	☐ Replay xác minh nghiệm cuối.
☐ So sánh tuned/baseline trên held-out.	☐ Ghi feature, lưới, tiêu chí, giới hạn; phân biệt harness với solver/GUI.